



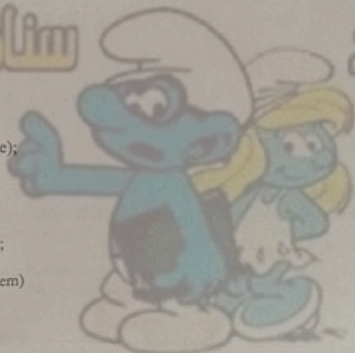
Problem # 4: If the C++ program below represents the implementation of the circular queue, write the output of this code. [5]

```
#include <iostream>
using namespace std;
const int size = 6;
template<class T>
class CircularQ
{
private:
    T q[size];
    int front, rear;
    int count;
public:
    CircularQ()
    {
        front = 0;
        rear = size - 1;
        count = 0;
    }
    bool isFull()
    {
        return(count==size);
    }
    bool isEmpty()
    {
        return (count==0);
    }
    void enqueue(T item)
    {
        if(isFull())
        {
            cout<<"full\n";
        }
        else
        {
            rear = (rear + 1) % size;
            q[rear] = item;
            count++;
        }
    }
    int dequeue()
    {
        if(isEmpty())
        {
            cout<<"empty";
            return 0;
        }
        else
        {
            int x = q[front];
            front = (front + 1) % size;
```

SMURFS POLYTECHNIC UNION

سمايفر

اتحاد

البوليتكنيك
SPU



8) Linked list allows random memory access.

A. True

B. False

9) Arrays are not cache friendly.

A. True

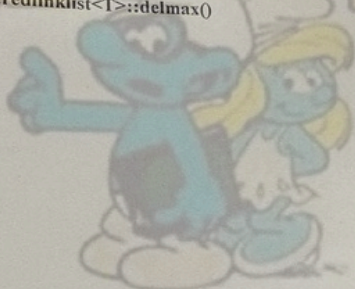
B. False

*** Write the correct answers of the problem # 1 in the table below:

<u>Q1</u>	<u>Q2</u>	<u>Q3</u>	<u>Q4</u>	<u>Q5</u>	<u>Q6</u>	<u>Q7</u>	<u>Q8</u>	<u>Q9</u>

Problem # 2: Write a C++ program to perform deletion to the node with the maximum number in a simple unordered linked list. [6pts]

```
template <class T>
T unorderedlinklist<T>::delmax()
{
```



البوليتكنك
SPU

}



Problem # 3: Write a program to explain class template by creating a template T for a class named **Minimum_value** having three data members (**num1**, **num2**, and **num3**) of type T which are inputted by a constructor and a member function **get-minimum()** return the minimum of the three numbers to main. Note: the value of T depends upon the data type specified during object creation. [6pts]

SMURFS POLYTECHNIC UNION

سمافر

اتحاد



البوليتكنيك

SPU



Problem # 5: Write the output of the below C++ code. [4]

```
#include <iostream>
using namespace std;
template <class T, class U, class V = char>
class ClassTemplate {
private:
    T var1;
    U var2;
    V var3;
public:
    ClassTemplate(T v1, U v2, V v3)
    {
        var1=v1;
        var2=v2;
        var3=v3;
    }
    void printVar() {
        cout << "var1 = " << var1 << endl;
        cout << "var2 = " << var2 << endl;
        cout << "var3 = " << var3 << endl;
    }
};
int main() {
    ClassTemplate<int, double> obj1(20, 19.8, 'd');
    cout << "obj1 values: " << endl;
    obj1.printVar();
    ClassTemplate<double, char, bool>
    obj2(12.8, 'c', true); cout << "nobj2 values: " << endl;
    obj2.printVar();
    return 0;
}
```

Output

Problem # 1: Multiple choice question. [8 x 1pt]

- 1) In the linear queue, the value of REAR is increased by 1 when
 - A. An element is deleted in a queue
 - B. An element is traversed in a queue
 - C. An element is added in a queue
 - D. An element is merged in a queue
- 2) The deletion operation in stack is called
 - A. insert
 - B. push
 - C. pop
 - D. enqueue
- 3) In singly linked list, link fields holds pointers to the element in the linked representation of stack.
 - A. Neighboring
 - B. Last
 - C. First
 - D. Middle
- 4) Which of the following is **FALSE** about Circular Queue?
 - A. It helps in refilling all the free spaces by resetting REAR to zero when the REAR reaches the end of the queue
 - B. It follows the Last in First out LIFO (LIFO) principle
 - C. It connects the last position of the queue to the first position of the queue
 - D. None of the above
- 5) Which of the following is **TRUE** about linear Queue?
 - A. It is also called Last-In-Last-Out list (LILO)
 - B. It becomes full when $\text{Front} = \text{size} - 1$
 - C. It becomes empty only when $\text{Rear} = -1$ and $\text{Front} = -1$.
 - D. None of the above
- 6) Which of the following is **incorrect** about stack?
 - A. If $\text{top} = -1$, then the stack is empty
 - B. If $\text{top} = -1$, then the stack is not empty
 - C. $\text{pop}()$ operation will remove (access) an element from the stack
 - D. None of the above
- 7) In doubly linked list, items can be navigated forward and backward.
 - A. True
 - B. False

```
count--;
return x;
}
} int getSize()
{
return count;
} void print_queue()
{
for (int i = front; i <= rear; i++)
cout << "Position : " << i << ", Value : " << q[i]<<endl;
}
};
```

```
int main()
{
CircularQ <int> qq;
qq.enqueue(72);
qq.enqueue(45);
qq.enqueue(17);
qq.enqueue(11);
qq.enqueue(2);
qq.enqueue(31);
qq.enqueue(9);
qq.enqueue(81);
for(int i=0; i<=qq.getSize()/2; i+=2)
    cout << qq.dequeue() << endl;
qq.print_queue();
cout<<qq.dequeue()<<"\n";
qq.enqueue(32);
qq.enqueue(35);
cout<<qq.dequeue();
return 0;
}
```

Output